Running time

Outline

Running time of an algorithm

Find maximum

Insertion sort

Change of base

Primality test

Summary

Where we were

- In the chapter «Input size» we learned how to measure the size $|I|$ of an input instance I
- $|I|$ is the **number of bits** needed to represent I
- In this chapter we use this measure to analyse algorithms

The central question

The central question of the analysis of algorithms:

how does the time needed by an algorithm grow when the input grows?

Plan of the chapter

- We define the running time of an algorithm, the primitive operations (**steps**) that it executes, and the **worst case**; we fix the conventions to count the steps of a pseudo-code line by line
- We apply the method to four algorithms already met in the course: `FindMaximum`, `InsertionSort`, `ChangeToBase2` and `PrimalityTest`
- For each of them: first the number of steps $S(n)$ as a function of a natural parameter n, then the order of growth as a function of the input size $|I|$
- Four different behaviours: linear, quadratic, linear in $|I|$ although logarithmic in n, exponential in $|I|$ although only $\Theta(\sqrt{n})$ in n; we compare them at the end

Running time of an algorithm

Introduction: Running time of an algorithm

- We define the running time of an algorithm and the primitive operations that it executes
- We fix the rules to count the steps of a pseudo-code, line by line
- We define the worst case, and the two steps of the analysis used in the whole chapter

Definition 1: running time

*The **running time** of an algorithm is an estimate of the time needed to execute it, as a function of the **input size** $|I|$.*

- We are not interested in the exact time in seconds, which depends on the computer, on the compiler and on many other details
- We are interested in the **order of growth** of the running time, expressed with the asymptotic notation O, Ω and Θ (chapter «Asymptotic notation», recalled in the chapter «Input size»)

What the running time depends on

The running time of an algorithm depends on the number of
primitive operations
executed by the algorithm on an input of a given size.

Primitive operations

Definition 2: primitive operation

*A **primitive operation** is a single operation that can be performed on a computer.*

The principal primitive operations are the following:

1. arithmetic operations on fixed-size numbers;
2. comparison of two fixed-size numbers;
3. assignment of a value to a variable;
4. accessing an element of an array by index;
5. control-flow operations (for example, evaluating a condition in an `if` statement, or iterating in a loop).

Constant time and steps

- We make the assumption that a **constant amount of time** is required to execute each primitive operation of an algorithm
- In this way the analysis is as machine-independent as possible
- For short, we call each primitive operation a **step**

Counting the steps of a pseudo-code

- Every line of our pseudo-codes executes a **constant** number of primitive operations (a few comparisons, arithmetic operations and assignments)
- Since we are interested in the order of growth, we count **one step for each execution of a line**: this changes the number of steps only by a constant factor
- To count the steps we determine, for every line, how many times it is executed, and we add up these numbers

The following conventions are used in the whole chapter (and in its exercises).

Line-by-line counting (1/2)

- Every execution of a numbered line is **one step**, including the lines `return`
- The condition of a `while` loop whose body is executed k times is evaluated $k + 1$ times: the last evaluation is the one that finds the condition false and ends the loop
- The condition of an `if` is evaluated every time the line is reached; the lines of the branch that is not taken are not executed

Line-by-line counting (2/2)

- When the number of executions of a line depends on the specific input (and not only on its size), we describe it with an **auxiliary function**, usually denoted by t
- Then we look for the input that makes the total number of steps as large as possible

Worst-case running time

- Even for inputs of a given size, the running time of an algorithm may depend on which specific input of that size is given
- Typically, in the analysis of algorithms, the **worst possible case** is considered, since it determines the longest running time for any input of a given size

Definition 3: worst-case running time

*The **worst-case running time** $T(|I|)$ of an algorithm is the maximum number of steps executed by the algorithm on any input I of a given input size $|I|$.*

Natural parameter and input size

- It is usually easier to count the steps as a function of a **natural parameter** of the input, such as
  - the number n of entries of an array, or
  - the value n of the input number
- The input size $|I|$, instead, is the number of bits of the input
- This leads to the method followed in all the examples of the chapter

The two steps of the analysis

1. Count the number of steps $S(n)$ executed in the worst case, as a function of the natural parameter n, and determine its order of growth (for example $S(n) = \Theta(n^2)$).

2. Determine the relation between n and the input size $|I|$ (for example $|I| = \Theta(n)$, or $|I| = \Theta(\log_2 n)$), and use it to express the order of growth of the worst-case running time $T(|I|)$.

The second step is not a formality

The same number of steps $\Theta(n)$ corresponds to a running time

linear in $|I|$ if n is the number of entries of an array,

and **exponential** in $|I|$ if n is the value of the input number.

Summary

Running time:

- An estimate of the time as a function of $|I|$, measured by its order of growth
- Measured in **steps**: primitive operations, each executed in constant time

Counting:

- One step for each execution of a line; a `while` condition is evaluated once more than its body
- The **worst case**: the maximum number of steps over the inputs of a given size

Two steps: first $S(n)$ in a natural parameter n, then $T(|I|)$ in the input size

Find maximum

Introduction: Find maximum

- Our first example: the simplest loop over an array
- We count the steps line by line, and we find the worst-case input
- Then we express the running time as a function of the input size, in the general case and with entries of a fixed number of bits

The problem

Given an array $a[1..n]$ of $n \geq 1$ integer numbers,
consider the problem of determining
the **maximum value** in the array.

- In the next slide, the pseudo-code of a simple algorithm, called `FindMaximum`, that solves this problem

Algorithm: FindMaximum

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output : The maximum value in array a

```
1  function FindMaximum(a, n)
2      max := a[1]
3      i := 2
4      while i ≤ n do
5          if a[i] > max then
6              max ← a[i]
7          end
8          i ← i + 1
9      end
10     return max
11 end
```

Run in Colab: Python · C++ · Java

Number of steps

- Let $S(n)$ denote the number of steps executed by `FindMaximum` in the worst case, where n is the size of the array
- The body of the loop is executed for $i = 2, 3, \ldots, n$, that is, $n - 1$ times, so the condition of line 4 is evaluated n times
- The execution of line 6, instead, depends on the iteration. We define a function

$$t : \{2, 3, \ldots, n\} \to \{0, 1\},$$

where $t(i) = 1$ if line 6 is executed at iteration i, and $t(i) = 0$ otherwise

Number of executions of each line

line	number of executions
2	1
3	1
4	n
5	$n-1$
6	$\sum_{i=2}^{n} t(i)$
8	$n-1$
10	1

Therefore, the total number of steps is

$$1+1+n+(n-1)+\sum_{i=2}^{n} t(i)+(n-1)+1 = 3n+1+\sum_{i=2}^{n} t(i).$$

The worst case

- The value of $t(i)$ depends on the specific input array, so we analyse the worst possible case
- The worst possible case is when the elements of the array are in **increasing order**:

$$a[1] < a[2] < \cdots < a[n]$$

- Then $t(i) = 1$ for every $i \in \{2, 3, \ldots, n\}$: we enter the `if` at every iteration, and we update the maximum value each time
- The total number of steps in the worst case is

$$3n + 1 + \sum_{i=2}^{n} 1 = 3n + 1 + (n - 1) = 4n.$$

The order of growth of $S(n)$

- Therefore,
$$S(n) = 4n = \Theta(n).$$
- Thus, $S(n)$ grows **linearly** in n. Indeed,
$$\lim_{n \to \infty} \frac{4n}{n} = 4 \in \mathbb{R}_{>0}.$$

Let us check the count on a small array.

Example (1/2)

Consider the array $a[1..4] = (3, 1, 4, 5)$.

- Iteration $i = 2$: $a[2] = 1 > 3$ is false, so $t(2) = 0$
- Iteration $i = 3$: $a[3] = 4 > 3$ is true, max becomes 4, so $t(3) = 1$
- Iteration $i = 4$: $a[4] = 5 > 4$ is true, max becomes 5, so $t(4) = 1$

Example (2/2)

- The algorithm returns 5 after

$$3 \cdot 4 + 1 + \big(t(2) + t(3) + t(4)\big) = 13 + 2 = 15$$

  steps

- On the increasing array $(1, 2, 3, 4)$, instead, $t(2) = t(3) = t(4) = 1$ and the algorithm executes

$$S(4) = 4 \cdot 4 = 16$$

  steps, the maximum for $n = 4$

Running time as a function of the input size

- We now determine the worst-case running time of the algorithm, by expressing the order of growth of the number of steps as a function of the input size $|I|$
- Two cases:
  - the **general case**, with no hypothesis on the entries
  - entries with a **fixed number of bits**

General case (1/2)

- The input consists of an array $a[1..n]$ of n integer numbers, so the input size satisfies

$$|I| = \Omega(n),$$

  because the input must contain at least the n array entries
- This means that, for some constant $c > 0$ and for all sufficiently large n,

$$|I| \geq c\, n.$$

- Equivalently,

$$n \leq \frac{1}{c}|I|,$$

  and hence $n = O(|I|)$

General case (2/2)

- Since $S(n) = \Theta(n)$, there exists a constant $a > 0$ such that

$$S(n) \leq a\, n$$

for all sufficiently large n

- Since $n \leq \frac{1}{c}|I|$, we obtain

$$S(n) \leq \frac{a}{c}|I|.$$

- Thus, the number of steps is bounded above by a constant times the input size

FindMaximum: general case

The worst-case running time of the `FindMaximum` algorithm, as a function of the input size $|I|$, is

$$T(|I|) = O(|I|).$$

Thus, in the general case, the worst-case running time is **at most linear** in the input size.

Entries with a fixed number of bits (1/2)

- We often make the hypothesis that all input values can be stored using a fixed constant number of bits, for example 32 bits
- Under this hypothesis, the input size satisfies

$$|I| = \Theta(n),$$

because the input consists of n array entries and each entry requires a constant number of bits
- This means that there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large n,

$$c_1 n \leq |I| \leq c_2 n.$$

- Equivalently,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|,$$

that is, $n = \Theta(|I|)$

Entries with a fixed number of bits (2/2)

- Since $S(n) = \Theta(n)$, there exist constants $a_1, a_2 > 0$ such that, for all sufficiently large n,

$$a_1 n \leq S(n) \leq a_2 n.$$

- Using the bounds on n in terms of $|I|$, we obtain

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|,$$

that is, $S(n) = \Theta(|I|)$

FindMaximum: entries with a fixed number of bits

If all input values use a fixed constant number of bits, then the worst-case running time of the `FindMaximum` algorithm is

$$T(|I|) = \Theta(|I|).$$

Thus, under this hypothesis,
the algorithm is **linear** in the input size.

Summary

Number of steps:

- $3n + 1 + \sum_{i=2}^{n} t(i)$, where $t(i) = 1$ if the maximum is updated at iteration i
- Worst case: increasing array, $S(n) = 4n = \Theta(n)$

Running time:

- General case: $|I| = \Omega(n)$, hence $T(|I|) = O(|I|)$
- Entries with a fixed number of bits: $|I| = \Theta(n)$, hence $T(|I|) = \Theta(|I|)$

Insertion sort

Introduction: Insertion sort

- In `FindMaximum` the loop body executes a constant number of steps
- In `InsertionSort` the body of the outer loop contains **another loop**, whose number of iterations depends on the input
- This produces a **quadratic** number of steps

The problem

Given an array $a[1..n]$ of n natural numbers,
consider the problem of sorting the array
in non-decreasing order:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- In the next slide, the pseudo-code of the `InsertionSort` algorithm

Algorithm: InsertionSort (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1 function InsertionSort(a, n)
2 |   i := 2
3 |   while i ≤ n do
4 |   |   key := a[i]
5 |   |   j := i − 1
```

Algorithm: InsertionSort (2/2)

```
function InsertionSort(a, n) (continued)
    while i ≤ n (continued) do
6       while j ≥ 1 and a[j] > key do
7           a[j + 1] ← a[j]
8           j ← j − 1
9       end
10      a[j + 1] ← key
11      i ← i + 1
12  end
13 end
```

Run in Colab: Python · C++ · Java

Number of steps

- Let $S(n)$ denote the number of steps executed by `InsertionSort` in the worst case, where n is the size of the array

- The execution of lines 7–8 depends on the iteration. We define a function

$$t : \{2, 3, \ldots, n\} \to \{0, 1, \ldots, n-1\},$$

  where $t(i)$ denotes the number of times the body of the inner loop (lines 7–8) is executed during the iteration i of the outer loop

- The condition of the inner loop (line 6) is then evaluated $t(i) + 1$ times during the same iteration

Number of executions of each line

line	number of executions
2	1
3	n
4	$n-1$
5	$n-1$
6	$(n-1)+\sum_{i=2}^{n} t(i)$
7	$\sum_{i=2}^{n} t(i)$
8	$\sum_{i=2}^{n} t(i)$
10	$n-1$
11	$n-1$

Total number of steps

Therefore, the total number of steps is

$$1 + n + (n-1) + (n-1) + \left((n-1) + \sum_{i=2}^{n} t(i) \right)$$

$$+ \sum_{i=2}^{n} t(i) + \sum_{i=2}^{n} t(i) + (n-1) + (n-1)$$

$$= 6n - 4 + 3 \sum_{i=2}^{n} t(i).$$

The worst case (1/2)

- The value of $t(i)$ depends on the specific input array, so we analyse the worst possible case
- Since j starts from $i-1$ and decreases down to 0 at most, we always have $t(i) \leq i-1$
- The worst possible case is when the elements of the array are in **strictly decreasing order**:

$$a[1] > a[2] > \cdots > a[n]$$

- In this case, at every outer iteration i, the element $key = a[i]$ is smaller than all elements before it. Hence the inner while loop runs to completion:

$$t(i) = i-1 \qquad \forall i \in \{2, 3, \ldots, n\}.$$

The worst case (2/2)

- Therefore,

$$\sum_{i=2}^{n} t(i) = \sum_{i=2}^{n} (i-1) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2}.$$

- The total number of steps in the worst case is

$$6n - 4 + 3 \cdot \frac{n(n-1)}{2} = \frac{12n - 8 + 3n^2 - 3n}{2} = \frac{3n^2 + 9n - 8}{2}.$$

The order of growth of $S(n)$

- Therefore,
$$S(n) = \frac{3n^2 + 9n - 8}{2} = \Theta(n^2).$$
- Thus, $S(n)$ grows **quadratically** in n. Indeed,
$$\lim_{n\to\infty} \frac{3n^2 + 9n - 8}{2n^2} = \lim_{n\to\infty} \frac{1}{2}\left(3 + \frac{9}{n} - \frac{8}{n^2}\right) = \frac{3}{2} \in \mathbb{R}_{>0}.$$

Let us see the function t on a concrete array.

Example (1/2)

- Consider the array $a[1..6] = (5, 2, 4, 6, 1, 3)$
- At each iteration i, $t(i)$ is the number of elements of $a[1..i-1]$ that are larger than $key = a[i]$, i.e., the number of elements shifted one position to the right

i	key	$t(i)$	array after the iteration
2	2	1	$(2, 5, 4, 6, 1, 3)$
3	4	1	$(2, 4, 5, 6, 1, 3)$
4	6	0	$(2, 4, 5, 6, 1, 3)$
5	1	4	$(1, 2, 4, 5, 6, 3)$
6	3	3	$(1, 2, 3, 4, 5, 6)$

Example (2/2)

- We have $\sum_{i=2}^{6} t(i) = 9$, hence the algorithm executes

$$6 \cdot 6 - 4 + 3 \cdot 9 = 59$$

steps

- On a decreasing array of 6 elements, instead, $\sum_{i=2}^{6} t(i) = 1 + 2 + 3 + 4 + 5 = 15$ and the algorithm executes

$$S(6) = \frac{3 \cdot 36 + 9 \cdot 6 - 8}{2} = 77$$

steps

Running time as a function of the input size

- We now determine the worst-case running time of the algorithm, by expressing the order of growth of the number of steps as a function of the input size $|I|$
- As for `FindMaximum`, two cases:
  - the **general case**
  - entries with a **fixed number of bits**

General case (1/2)

- The input consists of an array $a[1..n]$ of n natural numbers, so the input size satisfies

$$|I| = \Omega(n),$$

because the input must contain at least the n array entries

- This means that, for some constant $c > 0$ and for all sufficiently large n,

$$|I| \geq c\, n.$$

- Equivalently,

$$n \leq \frac{1}{c}|I|,$$

and hence $n = O(|I|)$

General case (2/2)

- Since $S(n) = \Theta(n^2)$, there exists a constant $a > 0$ such that

$$S(n) \leq a\, n^2$$

for all sufficiently large n

- Since $n \leq \frac{1}{c}|I|$, we obtain

$$S(n) \leq a \left(\frac{1}{c}|I|\right)^2 = \frac{a}{c^2}|I|^2.$$

- Thus, the number of steps is bounded above by a constant times $|I|^2$

InsertionSort: general case

The worst-case running time of the `InsertionSort` algorithm, as a function of the input size $|I|$, is

$$T(|I|) = O(|I|^2).$$

Thus, in the general case, the worst-case running time is **at most quadratic** in the input size.

Entries with a fixed number of bits (1/2)

- We often make the hypothesis that all input values can be stored using a fixed constant number of bits, for example 32 bits
- Under this hypothesis, the input size satisfies

$$|I| = \Theta(n),$$

  because the input consists of n array entries and each entry requires a constant number of bits
- This means that there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large n,

$$c_1 n \leq |I| \leq c_2 n.$$

- Equivalently,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

Entries with a fixed number of bits (2/2)

- Since $S(n) = \Theta(n^2)$, there exist constants $a_1, a_2 > 0$ such that, for all sufficiently large n,

$$a_1 n^2 \leq S(n) \leq a_2 n^2.$$

- Using the bounds on n in terms of $|I|$, we obtain

$$a_1 \left(\frac{1}{c_2}|I|\right)^2 \leq S(n) \leq a_2 \left(\frac{1}{c_1}|I|\right)^2,$$

that is,

$$\frac{a_1}{c_2^2}|I|^2 \leq S(n) \leq \frac{a_2}{c_1^2}|I|^2.$$

- Thus, $S(n) = \Theta(|I|^2)$

InsertionSort: entries with a fixed number of bits

If all input values use a fixed constant number of bits, then the worst-case running time of the `InsertionSort` algorithm is

$$T(|I|) = \Theta(|I|^2).$$

Thus, under this hypothesis,
the algorithm is **quadratic** in the input size.

Summary

Number of steps:

- $6n - 4 + 3\sum_{i=2}^{n} t(i)$, where $t(i)$ counts the iterations of the inner loop
- Worst case: strictly decreasing array, $t(i) = i - 1$, $S(n) = \frac{3n^2+9n-8}{2} = \Theta(n^2)$

Running time:

- General case: $T(|I|) = O(|I|^2)$
- Entries with a fixed number of bits: $T(|I|) = \Theta(|I|^2)$

Change of base

Introduction: Change of base

- In the first two examples the input is an array and the natural parameter n is the number of its entries
- We now consider an algorithm whose input is a **single number** n: the parameter n is now a value
- The relation between n and $|I|$ changes completely

The problem

Given a natural number $n \geq 1$ represented in base 10, consider the problem of converting n to base 2.

- The `ChangeToBase2` algorithm is the algorithm `ChangeOfBase` of the chapter «Numerical algorithms», with base $b = 2$
- The binary digits are stored in an array $d[0..nd-1]$ indexed from 0, so that $d[k]$ is the digit of weight 2^k

Algorithm: ChangeToBase2 (1/2)

Input : A natural number $n \geq 1$ in base 10
Output : An array d of binary digits and the number of digits nd

```
1 function ChangeToBase2(n)
2     nd := ⌊log2 n⌋ + 1
3     d := new array[0..nd − 1]
4     k := 0
```

Algorithm: ChangeToBase2 (2/2)

function *ChangeToBase2(n)* (continued)

```
5     while n > 0 do
6         q := ⌊n/2⌋
7         r := n mod 2
8         d[k] ← r
9         n ← q
10        k ← k + 1
11    end
12    return d, nd
13 end
```

Run in Colab: Python · C++ · Java

Number of steps (1/2)

- Let $S(n)$ denote the number of steps executed by `ChangeToBase2`, where n is the input value to be converted
- There is only one input for each value of n, so there is **no distinction** between the worst case and the other cases
- The number of digits needed to represent n in base 2 is

$$\lfloor \log_2 n \rfloor + 1.$$

- At every iteration of the while loop, the current value of n is replaced by $\lfloor n/2 \rfloor$. After s iterations, the current value is

$$\left\lfloor \frac{n}{2^s} \right\rfloor.$$

Number of steps (2/2)

- The loop stops when this value becomes 0, that is, when

$$\left\lfloor \frac{n}{2^s} \right\rfloor = 0.$$

- This happens exactly when $2^s > n$. The smallest integer s such that $2^s > n$ is

$$s = \lfloor \log_2 n \rfloor + 1.$$

- Therefore, the body of the while loop is executed exactly $\lfloor \log_2 n \rfloor + 1$ times (one iteration for each binary digit), and its condition is evaluated $\lfloor \log_2 n \rfloor + 2$ times

Number of executions of each line

line	number of executions
2, 3, 4	1 each
5	$\lfloor \log_2 n \rfloor + 2$
6, 7, 8, 9, 10	$\lfloor \log_2 n \rfloor + 1$ each
12	1

Therefore, the total number of steps is

$$1 + 1 + 1 + (\lfloor \log_2 n \rfloor + 2) + 5(\lfloor \log_2 n \rfloor + 1) + 1$$

$$= 6(\lfloor \log_2 n \rfloor + 1) + 5 = 6\lfloor \log_2 n \rfloor + 11.$$

The order of growth of $S(n)$ (1/2)

- Therefore,

$$S(n) = 6\lfloor \log_2 n \rfloor + 11 = \Theta(\log_2 n).$$

  In words, the number of steps grows **logarithmically** in the numerical value n

- Indeed, for $n > 1$,

$$\log_2 n - 1 < \lfloor \log_2 n \rfloor \leq \log_2 n.$$

- Dividing by $\log_2 n > 0$, we obtain

$$1 - \frac{1}{\log_2 n} < \frac{\lfloor \log_2 n \rfloor}{\log_2 n} \leq 1.$$

The order of growth of $S(n)$ (2/2)

- Since both $1 - \frac{1}{\log_2 n}$ and 1 converge to 1, by the squeeze theorem,

$$\lim_{n\to\infty} \frac{\lfloor \log_2 n \rfloor}{\log_2 n} = 1.$$

- Therefore,

$$\lim_{n\to\infty} \frac{6\lfloor \log_2 n \rfloor + 11}{\log_2 n} = \lim_{n\to\infty} \left(6 \frac{\lfloor \log_2 n \rfloor}{\log_2 n} + \frac{11}{\log_2 n} \right)$$

$$= 6 \cdot 1 + 0 = 6 \in \mathbb{R}_{>0}.$$

Example (1/2)

- Let us run `ChangeToBase2` on $n = 13$
- We have $\lfloor \log_2 13 \rfloor + 1 = 4$, so the array $d[0..3]$ is created and the loop executes 4 iterations:

iteration	n	$q = \lfloor n/2 \rfloor$	$r = n \bmod 2$	assignment
1	13	6	1	$d[0] = 1$
2	6	3	0	$d[1] = 0$
3	3	1	1	$d[2] = 1$
4	1	0	1	$d[3] = 1$

Example (2/2)

- After the fourth iteration $n = 0$: the condition of line 5 is evaluated for the fifth time, it is false, and the algorithm returns the digits of $13 = (1101)_2$
- The number of steps is

$$S(13) = 3 + 5 + 5 \cdot 4 + 1 = 29 = 6 \cdot 3 + 11.$$

Running time as a function of the input size (1/3)

- We now express the running time as a function of the input size $|I|$. This is the **delicate change of variable**
- The input is not the abstract value n, but a **representation** of n. Since n is represented in base 10, the number of digits needed to write n is

$$\lfloor \log_{10} n \rfloor + 1.$$

- Each decimal digit requires a constant number of bits. Hence the input size satisfies

$$|I| = \Theta(\lfloor \log_{10} n \rfloor + 1) = \Theta(\log_{10} n).$$

Running time as a function of the input size (2/3)

- Since the base of a logarithm changes it only by a constant factor ($\log_{10} n = \log_2 n / \log_2 10$), we get

$$|I| = \Theta(\log_2 n),$$

  the same order of growth obtained in the chapter «Input size» for a number written in binary
- This means that there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large n,

$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n.$$

- Equivalently,

$$\frac{1}{c_2}|I| \leq \log_2 n \leq \frac{1}{c_1}|I|.$$

Running time as a function of the input size (3/3)

- Since $S(n) = \Theta(\log_2 n)$, there exist constants $a_1, a_2 > 0$ such that, for all sufficiently large n,

$$a_1 \log_2 n \leq S(n) \leq a_2 \log_2 n.$$

- Using the bounds on $\log_2 n$ in terms of $|I|$, we obtain

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|,$$

that is, $S(n) = \Theta(|I|)$

ChangeToBase2: running time

The running time of the `ChangeToBase2` algorithm is

$$T(|I|) = \Theta(|I|).$$

Thus, the algorithm is **logarithmic** in the numerical value n,
but **linear** in the input size $|I|$,
because the input size is itself proportional to $\log_2 n$.

Summary

Number of steps:

- One iteration for each binary digit: $\lfloor \log_2 n \rfloor + 1$ iterations
- $S(n) = 6\lfloor \log_2 n \rfloor + 11 = \Theta(\log_2 n)$, the same for every input of value n

Running time:

- The input is a representation of n: $|I| = \Theta(\log_2 n)$
- Hence $T(|I|) = \Theta(|I|)$: logarithmic in n, linear in $|I|$

Primality test

Introduction: Primality test

- The input of the last example is again a single number n
- But now the number of steps grows as $\sqrt{n}$, much faster than $\log_2 n$
- We shall see that this makes the algorithm **exponential** in the input size

The problem

Given a natural number n,
consider the problem of determining
whether n is a **prime number**.

- In the next slide, the pseudo-code of the `PrimalityTest` algorithm of the chapter «Numerical algorithms»

Algorithm: PrimalityTest

Input : A natural number n
Output : `true` if n is prime, `false` otherwise

```
1  function PrimalityTest(n)
2      if n < 2 then
3          return false
4      end
5      d := 2
6      while d ≤ ⌊√n⌋ do
7          if n mod d = 0 then
8              return false
9          end
10         d ← d + 1
11     end
12     return true
13 end
```

Run in Colab: Python · C++ · Java

Number of steps (1/2)

- Let $S(n)$ denote the number of steps executed by `PrimalityTest` in the worst case, where n is the input value
- We assume $n \geq 2$, since for $n < 2$ the algorithm stops after 2 steps
- **The worst case is when n is prime.** In this case the algorithm never returns early at line 8, so the while loop runs to completion

Number of steps (2/2)

- The variable d takes the values

$$2, 3, \ldots, \lfloor \sqrt{n} \rfloor,$$

  hence the body of the while loop is executed $\lfloor \sqrt{n} \rfloor - 1$ times

- The condition of the while loop is evaluated once more, when $d = \lfloor \sqrt{n} \rfloor + 1$, so line 6 is executed $\lfloor \sqrt{n} \rfloor$ times

Number of executions of each line

line	number of executions (n prime)
2	1
3	0 (since $n \geq 2$)
5	1
6	$\lfloor \sqrt{n} \rfloor$
7	$\lfloor \sqrt{n} \rfloor - 1$
8	0 (since n is prime)
10	$\lfloor \sqrt{n} \rfloor - 1$
12	1

Therefore, the total number of steps in the worst case is

$$1 + 1 + \lfloor \sqrt{n} \rfloor + (\lfloor \sqrt{n} \rfloor - 1) + (\lfloor \sqrt{n} \rfloor - 1) + 1 = 3\lfloor \sqrt{n} \rfloor + 1.$$

This is indeed the worst case

- If $n \geq 2$ is not prime, let p be its smallest divisor larger than 1: then $p \leq \lfloor \sqrt{n} \rfloor$, and the algorithm returns `false` at line 8 when $d = p$
- Line 6 is executed $p - 1$ times, line 7 $p - 1$ times, line 10 $p - 2$ times, and lines 2, 5, 8 once each, for a total of

$$3p - 1 \leq 3\lfloor \sqrt{n} \rfloor - 1 < 3\lfloor \sqrt{n} \rfloor + 1$$

steps

The order of growth of $S(n)$ (1/2)

- Therefore,
$$S(n) = 3\lfloor\sqrt{n}\rfloor + 1 = \Theta(\sqrt{n}).$$
- Thus, $S(n)$ grows as the **square root** of n. Indeed,
$$\sqrt{n} - 1 < \lfloor\sqrt{n}\rfloor \leq \sqrt{n}.$$
- Dividing by $\sqrt{n} > 0$, we obtain
$$1 - \frac{1}{\sqrt{n}} < \frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} \leq 1.$$

The order of growth of $S(n)$ (2/2)

- Since both $1 - \frac{1}{\sqrt{n}}$ and 1 converge to 1, by the squeeze theorem,

$$\lim_{n\to\infty} \frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} = 1.$$

- Therefore,

$$\lim_{n\to\infty} \frac{3\lfloor\sqrt{n}\rfloor + 1}{\sqrt{n}} = \lim_{n\to\infty} \left(3\,\frac{\lfloor\sqrt{n}\rfloor}{\sqrt{n}} + \frac{1}{\sqrt{n}}\right)$$

$$= 3 \cdot 1 + 0 = 3 \in \mathbb{R}_{>0}.$$

Example (1/2)

- $n = 97$ is prime and $\lfloor \sqrt{97} \rfloor = 9$
- The algorithm tries the divisors $d = 2, 3, \ldots, 9$ (8 iterations), finds no divisor, and evaluates the condition of line 6 a ninth time with $d = 10$
- It returns `true` after

$$S(97) = 3 \cdot 9 + 1 = 28$$

steps

Example (2/2)

- $n = 91 = 7 \cdot 13$ is not prime, and its smallest divisor larger than 1 is $p = 7$
- The algorithm tries $d = 2, 3, \ldots, 6$ without success, and finds $91 \bmod 7 = 0$
- It returns `false` after

$$3 \cdot 7 - 1 = 20$$

steps, although $\lfloor \sqrt{91} \rfloor = 9$ as for 97

Running time as a function of the input size (1/4)

- We now determine the worst-case running time, by expressing the order of growth of the number of steps as a function of the input size $|I|$
- The input is a single natural number n. To represent n in binary, we need $\lfloor \log_2 n \rfloor + 1$ bits, so the input size satisfies

$$|I| = \Theta(\log_2 n).$$

- This means that there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large n,

$$c_1 \log_2 n \leq |I| \leq c_2 \log_2 n.$$

- Equivalently,

$$\frac{1}{c_2}|I| \leq \log_2 n \leq \frac{1}{c_1}|I|.$$

Running time as a function of the input size (2/4)

- The function $x \mapsto 2^x$ is strictly increasing. Hence, applying 2^x to all terms of the inequality preserves the order:

$$2^{\frac{1}{c_2}|I|} \leq 2^{\log_2 n} \leq 2^{\frac{1}{c_1}|I|}.$$

- Since $2^{\log_2 n} = n$, we obtain

$$2^{\frac{1}{c_2}|I|} \leq n \leq 2^{\frac{1}{c_1}|I|}.$$

- Moreover, $\sqrt{n} = n^{1/2}$. Taking the square root of all terms, which again preserves the order because all terms are positive, and using $(2^x)^{1/2} = 2^{x/2}$, we get

$$2^{\frac{1}{2c_2}|I|} \leq \sqrt{n} \leq 2^{\frac{1}{2c_1}|I|}.$$

Running time as a function of the input size (3/4)

- Since $S(n) = \Theta(\sqrt{n})$, there exist constants $a_1, a_2 > 0$ such that, for all sufficiently large n,

$$a_1\sqrt{n} \leq S(n) \leq a_2\sqrt{n}.$$

- Using the bounds on $\sqrt{n}$ in terms of $|I|$, we obtain

$$a_1\, 2^{\frac{1}{2c_2}|I|} \leq S(n) \leq a_2\, 2^{\frac{1}{2c_1}|I|}.$$

- Thus,

$$S(n) = 2^{\Theta(|I|)}.$$

Running time as a function of the input size (4/4)

- The worst case is attained on **prime numbers**, and there are prime numbers of every size
- For every $k \geq 2$ there is a prime number with exactly k binary digits, i.e., between 2^{k-1} and 2^k (Bertrand's postulate)
- Hence, for every input size, the maximum number of steps is attained by a prime n and it satisfies the bounds above

PrimalityTest: running time

The worst-case running time of the `PrimalityTest` algorithm, as a function of the input size $|I|$, is

$$T(|I|) = 2^{\Theta(|I|)}.$$

Accordingly, the order of growth of the worst-case running time of the `PrimalityTest` algorithm is **exponential** in the input size.

Summary

Number of steps:

- Worst case: n prime, the loop runs to completion
- $S(n) = 3\lfloor\sqrt{n}\rfloor + 1 = \Theta(\sqrt{n})$; a composite n with smallest divisor p takes $3p - 1$ steps

Running time:

- $|I| = \Theta(\log_2 n)$, hence $\sqrt{n} = 2^{\Theta(|I|)}$
- $T(|I|) = 2^{\Theta(|I|)}$: **exponential** in the input size

Summary

Introduction: Summary

- We compare the four analyses of the chapter in one table
- We collect the key points of the method

The four algorithms compared

- For the two algorithms on arrays, the running time is given under the hypothesis that all entries use a fixed constant number of bits

algorithm	parameter n	worst-case steps $S(n)$	input size $\|I\|$	running time $T(\|I\|)$
`FindMaximum`	entries of the array	$4n = \Theta(n)$	$\Theta(n)$	$\Theta(\|I\|)$
`InsertionSort`	entries of the array	$\frac{3n^2+9n-8}{2} = \Theta(n^2)$	$\Theta(n)$	$\Theta(\|I\|^2)$
`ChangeToBase2`	value of the number	$6\lfloor \log_2 n \rfloor + 11 = \Theta(\log_2 n)$	$\Theta(\log_2 n)$	$\Theta(\|I\|)$
`PrimalityTest`	value of the number	$3\lfloor \sqrt{n} \rfloor + 1 = \Theta(\sqrt{n})$	$\Theta(\log_2 n)$	$2^{\Theta(\|I\|)}$

Summary (1/3)

- The running time is measured in **steps** (primitive operations, each executed in constant time); we count one step for each execution of a line of pseudo-code
- We consider the **worst case**: the maximum number of steps over all inputs of a given size
- When the number of executions of a line depends on the input, an auxiliary function t describes it, and we look for the input that maximizes the total

Summary (2/3)

- The analysis has **two steps**: first the number of steps $S(n)$ as a function of a natural parameter n, then the running time $T(|I|)$ as a function of the input size
- If n is the number of entries of an array (with entries of fixed size), then $|I| = \Theta(n)$ and the order of growth does not change:

$$S(n) = \Theta(n^k) \quad \text{gives} \quad T(|I|) = \Theta(|I|^k)$$

Summary (3/3)

- If n is the **value** of the input number, then $|I| = \Theta(\log_2 n)$, i.e.,
$$n = 2^{\Theta(|I|)}$$
- A number of steps $\Theta(\log_2 n)$ is **linear** in $|I|$
- Any number of steps $\Theta(\sqrt{n})$ or $\Theta(n)$ is **exponential** in $|I|$